

Loops, Goals, and Iterations

Lecture 17 — Agents, Skills, and Hooks

Juan F. Imbet

Paris Dauphine – PSL University

June 26, 2026

- 1 From Linear Prompts to Iterative Systems
- 2 Goals as Control Objects
- 3 Agents, Skills, and Hooks
- 4 Designing an Iteration Loop
- 5 Example 1: Derivative-Free Root Finding
- 6 Example 2: Credit-Risk Stress-Testing Loop
- 7 Oversight, Failure Modes, and Patterns
- 8 Summary and Takeaways

The Prompt Is Not the Unit of Design — the Loop Is

THE BIGGER PICTURE

Earlier lectures treated the LLM as a *function*: prompt in, completion out. This lecture's thesis: almost every consequential use of an LLM in finance is a *process* — a sequence of model calls, tool calls, validations, and revisions around a goal the first call never achieves.

The methodology capstone of the course. We study the **design of iterative agentic systems**, not any single tool. Concrete tooling (Claude Code, Codex, the SDKs, Git) lives in Appendices A–F.

What you will leave with

A vocabulary — *goals, agents, skills, hooks, stopping rules* — and two worked loops: a transparent root-finder and a nine-agent credit-stress loop.

Why One-Shot Prompting Is Fragile

A one-shot prompt is an **open-loop controller**: output consumed downstream with *no error-correction channel*. The model never learns whether it was right. Fragility compounds along three axes:

- ❶ **Errors propagate without attenuation.** One forward pass to parse, extract, compute, narrate $\Rightarrow$ an early error corrupts all that follows, unnoticed.
- ❷ **The spec exceeds one pass.** Interacting constraints (turnover, sector caps, tracking error, tax lots) — joint satisfaction falls roughly *multiplicatively*.
- ❸ **The model cannot see the environment.** It guesses the schema, the rate, whether a file exists — acting on a *hallucinated* world.

THE BIGGER PICTURE

No analyst writes a DCF top-to-bottom and submits it. They sketch, sanity-check the terminal value, walk back an over-ceiling margin, re-run, circulate. The expert workflow is intrinsically *iterative*.

The Goal-Directed Loop

Five phases — repeat until $\text{ok}(s)$ holds or a stopping rule fires

Observe $\rightarrow$ Propose $\rightarrow$ Act $\rightarrow$ Evaluate $\rightarrow$ Revise

- **Observe** state s_t (files, tests, data, prior outputs).
- **Propose** action a_t toward goal G , conditioned on s_t .
- **Act**: execute $a_t \Rightarrow s_{t+1}$.
- **Evaluate**: feedback e_{t+1} from checks; does $\text{ok}(s_{t+1})$ hold?
- **Revise**: fold e_{t+1} into context, return to Observe.

The ordering is a dependency chain. **Revise feeds the concrete evidence of what went wrong back into the next Propose** — the error-correction channel one-shot prompting lacks.

The Evaluate phase carries most of the value

A loop is only as good as its feedback. A weak $\text{ok}(\cdot)$ converges happily to something useless. Design the feedback like you design the prompt.

Loops vs. Repetition — and When *Not* to Loop

UNDER THE HOOD

Genuine iteration: $a_{t+1} = \pi(s_{t+1}, e_{t+1}, h_t)$ — the next action depends on the *evaluated feedback*. If $a_{t+1} \perp e_{t+1}$, it is **mere repetition** (sampling), not iteration.

Anti-pattern — spurious repetition: re-prompt “fix the failing test” without reading whether it now passes. Same number of calls; *no information carried forward*; partial edits may leave the code worse.

Decision heuristic — loop only if: output is mechanically verifiable; reasoning varies with input; first attempt likely wrong *and* costly; no irreversible harm before a human gate.

Task	Right tool	Why
Bond yield to maturity	Pure function	Deterministic
Daily price-file download	Cron job	Scheduled, no reasoning
Figures from a novel filing	Loop + validator	Checkable output
Harden a valuation model	Agentic loop	Costly silent errors

Goals as Control Objects: Concrete, Checkable, Decomposable

The goal is the loop's **setpoint** — it governs behavior, decides when to stop, and decides what is acceptable. Three properties, one per phase:

- **Concrete** (for *Act*) — names the artifact. “Improve the model” → “raise OOS AUC of `credit.py`, measured by `backtest.py`, to ≥ 0.78 .”
- **Checkable** (for *Evaluate*) — supplies the predicate `ok(·)`: “every rubric dimension ≥ 90 via `/score-content`.”
- **Decomposable** (for *Revise*) — splits into independently checkable subgoals; localizes failures and bounds context.

A complete goal specifies *three* distinct things

Success criterion (stop because we won) · **Stopping rule** (stop anyway: budget, no-progress) · **Anticipated failure modes** (named in advance, each becomes a check). The common defect: specifying only the first.

Write the Goal as a File — Ideally Executable

A goal in a chat prompt vanishes when context rolls over. A goal in a *file* is durable, auditable, version-controlled, and — when executable — usable **directly as the Evaluate phase**.

```
# TASK: Harden the FCF valuation script
## Artifact
valuation/dcf.py (value_company(ticker) -> float)
## Success criteria
- pytest tests/test_dcf.py exits 0
- Coverage of dcf.py >= 90%
- No division-by-zero on the 12 edge-case tickers
## Stopping rule
- Halt after 6 iterations, or 2 with no new passing tests.
## Anticipated failure modes
- Passing tests by catching all exceptions and returning 0
  -> forbidden; every value finite & positive or raise ValueError.
```

The decisive move: make “done” and “acceptable” *the same code* — a `validate.sh` that exits 0 iff the goal is met, wired into Evaluate and enforced before commit.

Bad Goals vs. Good Goals — a Diagnostic

Bad goal	Good goal	Repair adds
“Improve the chapter.”	Every dimension ≥ 90 via /score-content.	Checkable predicate
“Make the model accurate.”	Held-out AUC ≥ 0.78 on 2023; no post-origination features.	Artifact, target, leakage guard
“Get tests passing.”	pytest 0 and coverage $\geq 90\%$; swallowing exceptions forbidden.	Anti-gaming guard
“Keep improving the portfolio.”	Max Sharpe s.t. turnover $\leq 30\%$; stop after 8 runs or gain < 0.05 .	Constraint + stopping rule

Each bad goal disables a loop phase

Unconcrete, uncheckable, or unbounded. **A loop is only ever as good as the goal you hand it** — no amount of clever iteration recovers a broken goal.

Agents as Specialized Iterators — Five Roles

Useful agents are not chatbots-with-tools; they are **specialized iterators**, each doing one transformation on project state and handing off.

- **Worker** — produces/transforms the artifact (the only mandatory role).
- **Critic** — structured judgment without rewriting; the natural-language *gradient*.
- **Validator** — objective deterministic pass/fail; the role most often omitted and most missed.
- **Planner** — decomposes a goal into ordered subtasks.
- **Refactorer** — improves structure, preserves behavior.

THE BIGGER PICTURE

Critic vs. validator, in finance terms: the critic is the senior reviewer who says the model *looks* aggressive; the validator is the limit system that simply *rejects* the trade. A loop of only workers + critics can drift confidently in a wrong direction.

Central discipline: an agent's value is the *durable change to state* (files, green tests, a commit) — not the text it prints. Measure it by its diff.

Skills: Reusable Procedures (a Skill $\approx$ a Function)

If agents are the *who*, a skill is the *how*: a named, composable sequence run the same way every time (`/draft-chapter`, `/score-content`, `/iterate-book-quality`).

The skill–function correspondence

A *name* you invoke it by · *scoped inputs* · a *single responsibility* · *repeatable* (same inputs $\rightarrow$ same procedure) · *composable* (one skill calls another: `/draft-chapter` $\rightarrow$ `/score-content` $\rightarrow$ `/refine-until-threshold`).

Two further payoffs.

- **Freeze conventions:** `/new-topic` creates chapter/lecture/notebook *N* together — numbering *cannot* drift.
- **Reduce entropy:** every invocation pushes a long-running project *back* toward consistency. The skill library is the project's control framework — its standard operating procedures.

Hooks: Guardrails That Do Not Depend on the Agent

A **hook** runs automatically on an event (pre-tool, post-write, session-end), **outside the agent's control**. Its defining property: it does not depend on the agent's cooperation.

Pre-action = pre-trade limit check (can block). **Post-action** = post-trade surveillance (validate / record / repair / escalate).

```
#!/usr/bin/env bash
# check-citations.sh -- post-write validation hook
set -euo pipefail
BIB="book/bibliography.bib"; MISSING=0
for f in $(git diff --cached --name-only | grep '\.tex$'); do
    for key in $(grep -oE '\\cite\[([^\]]+)\]' "$f" \
        | sed -E 's/\\cite\[([^\]]+)\]/\1/'); do
        grep -q "${key}," "$BIB" || { echo "Unresolved: $key"; MISSING=$((MISSING
+1)); }
    done
done
[ "$MISSING" -gt 0 ] && { echo "FAIL"; exit 1; }; echo "OK"; exit 0
```

A hook converts “is this good?” (opinion) into “compiles, citations resolve, scores $\geq$ threshold” (a *reproducible bit*). Hang the stopping condition on the bit, not on the model's confidence.

Trustworthy Validators and Failure Hooks

UNDER THE HOOD

A passing check is necessary, not sufficient. The loop optimizes for $V = \text{pass}$, not the true goal G . Unless $\{x : V(x) = \text{pass}\} \subseteq \{x : G \text{ met}\}$, there exist artifacts that pass yet fail — and an optimizing loop will find them (Goodhart).

Make V hard to satisfy without meeting G : outcome-tied not proxy checks; *diverse* checks (game all at once); validator logic out of the worker's reach; a hidden release-time holdout. (Model-risk management's playbook.)

Failure hooks — what fires on a red verdict (escalating)

Block (refuse) · **Revert** (undo to last-good) · **Escalate** (hand to a human) · **Re-enter** (feed the failure back as next input). The operational-risk playbook in miniature.

The Seven-Step Template

- 1 **Read state** — ground truth from disk + version control first.
- 2 **State the next local goal** — a checkable target for *this* turn.
- 3 **Propose** a change (reason).
- 4 **Apply** it (act — write to disk; only now has the world changed).
- 5 **Validate** — a deterministic check; the agent does *not* grade its own work.
- 6 **Interpret feedback** — direction and magnitude of the error.
- 7 **Decide**: stop / continue / **escalate**.

Two non-negotiables

Validation (5), interpretation (6), and decision (7) are *separate* — collapsing them removes the independent check. And the decision always includes **escalate**: a loop that can only succeed or grind forever *will* grind forever.

Pseudocode — and Its Map onto Claude Code Primitives

```
state <- read_state()           # file reads
goal  <- read_goal()           # checked-in goal file
best  <- evaluate(state, goal)
while not goal.satisfied(best) and not goal.budget_exhausted():
  local  <- choose_objective(state, best)
  change <- propose(state, local) # REASON (ReAct)
  state2 <- apply(change, state)  # ACT: write to disk
  report <- validate(state2)      # a SKILL: deterministic check
  if report.improves_on(best):
    commit(state2, summarize(change, report)) # audit trail
    state, best <- state2, report.score       # accept
  else:
    state <- rollback(state)             # reject (Self-Refine)
  if stalled(history) or report.requires_human:
    escalate(state, report); break       # hook-gated
emit_report(state, best, history)        # hook-gated
```

The accept/reject discipline makes the loop **robust to a noisy proposer**: a bad proposal costs one iteration, not the whole run — because validation and version control are deterministic.

State Lives on Disk; Stop on a Budget

THE BIGGER PICTURE

Design for amnesia. The agent has no durable memory between turns — it is a new employee every morning who reads the files, does a day's work, commits it, and is replaced overnight. **Write before you forget.**

Persists: files, git history (long-term memory), validation reports, a human-readable status scratchpad. **Lost:** chain-of-thought, unsaved values. Bonus: all-on-disk state makes the loop *resumable*.

Stopping rules — exactly one must be guaranteed-terminating

Success · convergence (no gain for k steps) · **budget exhaustion (mandatory)** · human-review-required. Convergence/escalation make the loop *smart*; the budget makes it *safe*.

Economics: marginal value shrinks near the goal while cost stays flat — stop where marginal value crosses marginal cost (latency binds interactively; dollars bind overnight).

Example 1: A Loop That Improves Using Feedback Alone

Find x with $f(x) = x^3 - x - 2 \approx 0$ to tolerance $|f(x)| < 10^{-3}$ (single root near $x \approx 1.52$).

The point is not numerical analysis. The agent may *evaluate* f anywhere but is **forbidden derivatives** — no $f'(x) = 3x^2 - 1$, no Newton update. It reads only the sign and size of the residual. This is the whole machinery, stripped bare.

THE BIGGER PICTURE

Finance analogue: this is mechanically an *IRR* or *implied-volatility* solver. For an IRR, seek r with $\text{NPV}(r) = \sum_t c_t / (1+r)^t = 0$ — no closed form, so evaluate at trial rates and move by the sign (too-high NPV $\Rightarrow$ rate too low). Swap f and the loop is unchanged.

Three artifacts, none containing an algorithm: a *goal file* ($|f| < 0.001, \leq 12$ iters), a *skill* (`python evaluator.py $x | tee -a results.log`), and a *stop hook* that vetoes “success” unless the last $|f(x)| < 10^{-3}$.

Iteration Trace from a Deliberately Poor Start ($x = 0$)

Iter	x	$f(x)$	Agent's reasoning
1	0	-2	Negative; root to the right. Jump up.
2	2	4	Positive; root bracketed in $(0, 2)$.
3	1.5	-0.125	Negative but small; root just above 1.5.
4	1.6	0.496	Positive; bracket $(1.5, 1.6)$.
5	1.52	-0.008192	Tiny negative; nudge up.
6	1.522	≈ 0.004	Overshot; back off a touch.
7	1.5214	≈ 0	$ f(x) < 0.001$: hook passes.

The **sign change** between 0 and 2 is the engine: a continuous function going $- \rightarrow +$ crosses zero between. The magnitudes ($|f(1.5)| \ll |f(1.6)|$) place the root near 1.5 (false-position reading). **Declaring success at iteration 5 would be vetoed by the hook.**

Why the inefficiency is the point

The zig-zag past the root means the loop reached tolerance *without an algorithm* — so the same loop works when the residual is a failing test, a style violation, or a reviewer's objection.

Example 2: A Defensible Credit-Committee Memo

Meridian Components (fictional; all figures illustrative): a mid-market manufacturer seeking a \$40 m revolver. Looks comfortable (growth, rising EBITDA, leverage in appetite) — but cyclical end markets, illiquid inventory, concentrated receivables.

The goal is *not* “estimate the PD”

A point estimate cannot be *wrong* — nothing fails against it. The goal is a *memo a committee would accept*: that acceptance is checkable, so the loop runs until the checks pass.

The expected-loss decomposition (reconciled in every column):

$$EL = PD \times LGD \times EAD.$$

PD = 1-yr default prob.; LGD = 1 – recovery; EAD = drawn balance plus drawdown. The memo must give PD, LGD, EAD, EL (\$ and bps), drivers, and a recommendation — *lend / modify / decline*. A “low risk” phrase is gated on both base *and* stressed EL clearing thresholds.

The Combined Shock and the Nine-Agent Ensemble

Combined macro shock (parameter table, not prose):

Parameter	Adv.	Sev.
Revenue	−20%	−30%
Gross margin	−300	−450 bps
Rate	+250	+350 bps
DSO	+25	+40 d
Inventory days	+20	+35 d
ICR covenant	→ 2.5×	

Combined, because the drivers are correlated; breaches trigger only when several coincide.

Nine role-scoped agents (each output → next input):

- 1 Data-ingestion
- 2 Accounting-quality (normalize)
- 3 Feature-engineering
- 4 PD-model *(leakage hook)*
- 5 Stress-scenario
- 6 Cash-flow (covenants, liquidity)
- 7 LGD (collateral, recovery)
- 8 **Skeptic** — returns to ANY agent
- 9 Memo-writing *(skeptic-gated)*

Two *feedback edges* (skeptic + failing hooks) turn a pipeline into a loop.

The two load-bearing hooks

Leakage check (the easy path to a beautiful backtest is to peek at the answer) and the **“low risk” gate** (language is a control surface — earned by arithmetic, not chosen by the writer).

Eleven Iterations, One Exhibit (EAD = \$40 m)

The arc:

- 1 Naive PD $\approx 1.2\%$
- 2 Add-backs $\Rightarrow$ lev. 3.0 $\rightarrow 3.8\times$
- 3 Richer features
- 4 **Leakage hook rejects** PD 0.9%
- 5 Clean rebuild $\rightarrow$ PD 2.5%
- 6 Adverse stress $\rightarrow$ PD 9%
- 7 Covenant breach; $\sim$ \$6 m hole
- 8 LGD 40% $\rightarrow 55\%$
- 9 Skeptic demands severe + peers
- 10 Severe: PD 13%, LGD 60%
- 11 Conditional approval

Metric	Base	Stressed
PD	2.5%	9.0%
LGD	40%	55%
EAD	\$40.0 m	\$40.0 m
EL (\$)	\$0.40 m	\$1.98 m
EL (bps)	100	495
Covenant breach?	No	Yes
Liquidity short.	—	$\approx$ \$6.0 m

$$EL_{\text{base}} = 0.025 \cdot 0.40 \cdot 40 = \$0.40 \text{ m};$$

$$EL_{\text{str}} = 0.09 \cdot 0.55 \cdot 40 = \$1.98 \text{ m}.$$

Action: conditional — \$30 m, tighter covenants, more collateral, repriced.

The story the loop found, the naive memo missed

Sound *in expectation*, fragile *in the tail*: EL nearly quintuples, and the damage is *structural* (breach + liquidity hole). A 1.2% point estimate hid all of it. A *recommendation*, not a decision — a human signs.

What does the signature add? Hooks answer “did it pass the checks?”, never “who is answerable?” The loop manufactures analysis, not accountability or legal standing.

- **Escalation triggers** (stated in the goal file, not improvised): materiality threshold crossed, low confidence / out-of-range inputs, agents that cannot reconcile, a hook firing repeatedly, irreversible action. *Human-review-required* is a terminal state.
- **Sign-off** (SR 11-7) = an event with time, identity, scope — not a property of the artifact. The skeptic is internal challenge, **not** independent validation. Carry an *unsigned sign-off* block the loop cannot complete.
- **Audit trail:** version control + structured run log + content-addressed inputs $\Rightarrow$ an auditor can *regenerate* every number. “Reproducible” means the derivation is recoverable and therefore challengeable.

Failure Modes — and the Control That Catches Each

Failure mode	Warning sign	Control
Infinite loop	Iters climb, no change; oscillation	Hard budget + cost ceiling
Fake convergence	“Done” but metric plateaued below goal	Terminate on <i>external</i> validator, not self-report
Goal drift	Later iters optimize a different objective	File-based goal re-read each iteration
Premature success	“Low risk” before the number is met	Tolerance hook
Goodhart overfit	One check passes; OOS lags	Diverse + held-out + adversarial
Excessive delegation	Agents for deterministic subtasks	Functions for closed-form; when-not-to-loop test

The meta-point

Every control is a mechanism introduced earlier. **The failure modes are the predictable consequences of omitting machinery the lecture already argued for** — build the loop right and you have defended against most of them by construction.

Five Design Patterns

- **Planner–executor–critic** — planning + execution + independent evaluation; for tasks where the next step depends on earlier results.
- **Generate–test–revise** — the tightest loop; default when the goal is “make this check pass.” Only as good as its test.
- **Scaffold–draft–audit–repair** — for subjective, rubric-scored goals. **This is the pattern behind this very book.**
- **Data-pipeline: build–validate–document** — the artifact is a process; *lineage* must be trustworthy.
- **Credit-model: build–stress-test–red-team–finalize** — Example 2 generalized; stress-test and red-team are *mandatory* stages (SR 11-7 governance encoded as a pattern).

Choosing — and composing

Pick by the goal’s properties (machine-checkable vs. judgment; number vs. artifact; stakes; reproducibility). Real workflows *nest* patterns. What never changes: the feedback must be **informative**.

Lecture 17 Summary

The shift: from issuing instructions and accepting the reply, to **designing a goal-directed loop** whose output is observed, evaluated, and revised against an explicit objective.

Three building blocks: *agents* specialize the loop (*who*), *skills* reuse it (*how*), *hooks* enforce it (*what it may never violate*).

Three controls: goals (the setpoint), validators (honest feedback), stopping rules (terminate safely).

The one idea to internalize

A loop adds value *exactly to the degree each iteration's feedback informs the next action*. In finance the goal is never to produce an answer, but to produce an answer someone can stand behind — achieved not by a better prompt but by a **better loop**.

Next (practical session): hand-run the root-finding loop, write a tolerance success-gate in bash, and decide lend/no-lend from base vs. stressed EL.

APPENDIX

Definitions, Literature, and Reference Material

Extra / optional material — beyond the core lecture.

Reference: The Agentic-AI Lineage

LLM side — the loop generalizes a family of methods

ReAct (Yao 2022): interleave reason + act = Observe–Propose–Act. **Reflexion** (Shinn 2023): verbalized Evaluate–Revise. **Self-Refine** (Madaan 2023): single-model generate–critique–revise. **Chain-of-thought** (Wei 2022); **Toolformer** (Schick 2023); **AutoGen** (Wu 2023): split phases across role-typed agents.

Engineering side — the loop predates LLMs by decades

Boehm's **spiral model** (1988): risk-driven cycles replace waterfall. Deming's **PDCA**: Plan–Do–Check–Act. The convergence is no coincidence: it is what any system producing correct output under uncertainty looks like once it closes its feedback loop.

Control-theoretic reading: one-shot = open-loop; goal-directed = feedback control. Error $e_{t+1} = G - s_{t+1}$; instability when gain is too high (oscillation); steady-state error when the success predicate is biased.

Reference: The Meridian Acceptance Criteria

Deliverable	Acceptance criterion	Checked by
PD (base & stressed)	OOS-validated; horizon stated	Hook + skeptic
LGD	Tied to collateral; recovery logic	Skeptic
EAD	Drawdown assumption stated	Hook
Expected loss	$EL = PD \cdot LGD \cdot EAD$ reconciles	Hook
Risk drivers	Each traced to a source statement	Hook
Scenarios	Base / adverse / severe, parameterized	Hook
Recommendation	lend / modify / decline, justified	Skeptic
"Low risk"	Only if base <i>and</i> stressed EL below thresholds	Hook

The test for "goal" vs. "task"

Ask what it would mean for the work to be *wrong*. "Estimate the PD" cannot be wrong. "A memo whose stressed EL reconciles, whose every ratio ties to source, and which the skeptic cannot break" can be wrong in a dozen concrete ways — and each is a hook waiting to fire.

Reference: The Root-Finding Stop Hook

```
#!/usr/bin/env bash
# stop-hook: veto "success" unless the latest  $|f(x)| < 0.001$ 
last=$(grep -o 'f(x)=[-0-9.eE]*' results.log | tail -n1 | cut -d= -f2)
abs=$(python -c "print(abs(float('$last')))" )
ok=$(python -c "print(1 if abs(float('$last')) < 0.001 else 0)" )
if [ "$ok" -ne 1 ]; then
    echo "BLOCKED:  $|f(x)| = $abs \geq 0.001$  -- not converged" >&2
    exit 1
fi
echo "OK:  $|f(x)| = $abs < 0.001$ "
exit 0
```

The harness treats a nonzero exit from a stop hook as a *veto*: the agent literally cannot end the loop claiming success while the residual is too large. The goal says *what* convergence means; the skill says *how* to measure; this hook *enforces* it.